

TG — Prompt Consolidado para Claude Code

Sistema Imersivo de Modelagem e Simulação de Sistemas Espaciais em Ambiente Virtual

Documento técnico preparado para Victor Stein

2026-05-21

Table of Contents

Documento técnico consolidado

Finalidade deste PDF

Este documento consolida o escopo técnico, a fundamentação de Engenharia de Sistemas, a arquitetura de software, o modelo de dados e um **prompt final pronto para copiar e colar no Claude Code**.

O objetivo é fazer o Claude Code implementar um MVP complexo, mas ainda executável, para o TG:

Sistema Imersivo de Modelagem e Simulação de Sistemas Espaciais em Ambiente Virtual

A aplicação resultante deve carregar uma arquitetura espacial definida em JSON e transformá-la em uma visualização interativa 2D/3D, com rastreabilidade entre missão, objetivos, requisitos, funções, subsistemas, componentes, interfaces, budgets, riscos e verificações.

1. Ideia central do projeto

O projeto deve demonstrar que uma arquitetura de sistema espacial pode ser compreendida melhor quando representada como uma rede interativa de entidades de Engenharia de Sistemas.

O sistema espacial não deve ser tratado apenas como uma lista de peças. Ele deve ser tratado como uma estrutura rastreável:

Missão — Objetivos — Requisitos — Funções — Sistema — Segmentos — Subsistemas — Componentes — Interfaces — Budgets — Riscos — Verificação — Validação

Essa cadeia precisa aparecer no software por meio de:

1. nós de diferentes tipos;
 2. relações entre nós;
 3. rastreabilidade visual;
 4. visualização progressiva por camadas;
 5. painel de detalhes;
 6. matriz de verificação;
 7. budgets técnicos;
 8. riscos técnicos;
 9. visualização 3D conceitual.
-

2. Fundamentação técnica resumida

2.1 Engenharia de Sistemas

A Engenharia de Sistemas é o processo técnico que permite transformar necessidades e objetivos de missão em uma solução implementável. Para este TG, a cadeia adotada será:

1. entender a missão;
2. identificar stakeholders e objetivos;
3. derivar requisitos;
4. decompor funções;
5. definir arquitetura;
6. alocar funções a subsistemas e componentes;
7. controlar interfaces;
8. acompanhar budgets;
9. gerenciar riscos;
10. verificar atendimento aos requisitos;
11. validar aderência à necessidade de missão.

A aplicação não deve tentar substituir ferramentas industriais completas de MBSE. Ela deve demonstrar, em um MVP, o princípio de que arquiteturas espaciais podem ser exploradas visualmente a partir de um modelo de dados estruturado.

2.2 Rastreabilidade

A rastreabilidade é central para o TG. Cada requisito deve poder ser ligado a:

- objetivo de missão;
- função;
- subsistema ou componente;
- interface relevante;

- método de verificação;
- evidência esperada.

Exemplo de cadeia:

MIS-EARTH-OBS-001

→ OBJ-IMG-001

→ REQ-IMG-001

→ FUN-CAPTURE-IMAGE

→ CMP-CAMERA

→ VER-REQ-IMG-001

O MVP deve permitir visualizar esse caminho no painel de rastreabilidade.

2.3 Arquitetura espacial de referência

O JSON demo deve representar uma missão simples de observação da Terra em órbita baixa, contendo:

- missão;
- objetivos;
- requisitos;
- funções;
- segmento espacial;
- segmento solo;
- satélite;
- estação solo;
- payload óptico;
- bus/service module;
- estrutura;
- EPS;
- OBC/OBDH;
- TT&C;
- ADCS;
- controle térmico;
- software embarcado;
- interfaces;
- budgets;
- riscos;
- verificações;
- operações.

2.4 Payload e bus

O satélite deve ser representado como uma composição de:

- **Payload:** motivação principal da missão, por exemplo câmera óptica.
- **Bus ou service module:** suporte ao payload, incluindo potência, comunicação, processamento, estrutura, térmico e atitude.

Funções típicas do bus no JSON demo:

- fornecer energia;
- controlar atitude;
- comunicar dados e comandos;
- manter estrutura;
- manter faixa térmica;
- executar software embarcado;
- armazenar e processar dados.

2.5 Verificação e validação

Use a distinção:

- **Verificação:** confirma se o produto atende aos requisitos especificados.
- **Validação:** confirma se o produto atende à necessidade real da missão e ao uso pretendido.

Métodos de verificação permitidos:

- Test
- Analysis
- Inspection
- Review of Design
- Demonstration

A matriz de verificação deve responder:

Qual requisito está sendo verificado?

Por qual método?

Qual item está sendo verificado?

Qual evidência é esperada?

Qual é o status?

2.6 AIT/AIV

O MVP deve representar AIT/AIV de forma conceitual:

- Assembly;
- Integration;
- Test;
- Verification;
- Validation.

Não é necessário modelar uma campanha de testes real. Basta mostrar entidades de verificação/teste e sua relação com requisitos e itens físicos.

2.7 Product Assurance e riscos

Product Assurance deve aparecer no MVP em nível conceitual via:

- riscos técnicos;
- status de verificação;
- confiabilidade conceitual;
- controle de configuração como campo/metadado;
- evidência de verificação;
- maturidade de modelo.

Riscos mínimos no demo:

- potência insuficiente;
 - perda de comunicação;
 - temperatura fora da faixa;
 - falha de apontamento;
 - perda de dados.
-

3. Escopo do MVP

3.1 O que deve ser implementado

O MVP deve:

1. inicializar ou ajustar projeto Vite + React + TypeScript;
2. carregar automaticamente um JSON demo;
3. validar o JSON com Zod;
4. mostrar status de validação;
5. transformar JSON em grafo 2D com React Flow;
6. transformar JSON em cena 3D com React Three Fiber;
7. permitir seleção de entidades;
8. exibir painel de detalhes;
9. exibir rastreabilidade;
10. exibir budgets;
11. exibir verificações;
12. exibir riscos;
13. exibir interfaces;
14. permitir construção progressiva por camadas;
15. alternar entre modo 2D e 3D;

16. compilar sem erros com npm run build.

3.2 Fora do escopo

Não implementar:

- backend;
- banco de dados;
- login;
- autenticação;
- simulação orbital real;
- propagação orbital;
- dinâmica de atitude real;
- controle real;
- modelo térmico físico;
- FEA;
- CFD;
- VR completo;
- integração com hardware;
- cálculo real de link budget;
- cálculo real de power budget;
- cálculo industrial de massa.

Esses itens podem ser citados como trabalhos futuros na documentação do README, mas não devem entrar no MVP.

4. Stack obrigatória

A stack deve ser:

Vite

React

TypeScript

@xyflow/react

three

@react-three/fiber

@react-three/drei

zod

zustand

lucide-react

CSS puro

Evitar Tailwind para reduzir risco de configuração. Usar CSS puro em src/index.css.

Comandos esperados:

```
npm install  
npm run dev  
npm run build
```

Se o projeto ainda não estiver inicializado:

```
npm create vite@latest . -- --template react-ts
```

Instalação de dependências:

```
npm install @xyflow/react three @react-three/fiber @react-three/drei zod zustand lucide-react
```

5. Estrutura de arquivos

A estrutura esperada é:

```
src/  
  App.tsx  
  main.tsx  
  index.css  
  
data/  
  demoArchitecture.json  
  
types/  
  architecture.ts  
  
schemas/  
  architectureSchema.ts  
  
store/  
  useArchitectureStore.ts  
  
lib/  
  validateArchitecture.ts  
  progressiveBuild.ts  
  architectureToFlow.ts  
  architectureToScene.ts  
  traceability.ts  
  formatters.ts
```

```
components/  
  layout/  
    AppShell.tsx  
    Header.tsx  
    Sidebar.tsx  
  
  flow/  
    ArchitectureFlow.tsx  
    CustomNode.tsx  
    nodeStyles.ts  
  
  scene3d/  
    ArchitectureScene3D.tsx  
    SpaceSystemObject.tsx  
    InterfaceLine3D.tsx  
  
  panels/  
    DetailsPanel.tsx  
    TraceabilityPanel.tsx  
    BudgetPanel.tsx  
    VerificationPanel.tsx  
    RiskPanel.tsx  
  
  controls/  
    ViewModeToggle.tsx  
    BuildControls.tsx  
    JsonValidationStatus.tsx
```

6. Modelo de datos

6.1 Entidades

Tipos mínimos:

```
export type EntityType =  
  | "mission"  
  | "objective"  
  | "requirement"  
  | "function"  
  | "system"  
  | "segment"  
  | "subsystem"
```



```
| "component"  
| "interface"  
| "verification"  
| "test"  
| "risk"  
| "budget"  
| "operation"  
| "model";
```

6.2 Relações

Tipos mínimos:

```
export type RelationType =  
| "contains"  
| "satisfies"  
| "allocated_to"  
| "verifies"  
| "validates"  
| "depends_on"  
| "provides_power_to"  
| "sends_data_to"  
| "receives_command_from"  
| "mechanically_attached_to"  
| "thermally_coupled_to"  
| "controls"  
| "measures"  
| "actuates"  
| "communicates_with"  
| "mitigates"  
| "constrains";
```

6.3 Interface principal

```
export interface ArchitectureEntity {  
  id: string;  
  name: string;  
  type: EntityType;  
  parentId?: string;  
  description?: string;  
  
  requirements?: string[];  
  functions?: string[];  
  interfaces?: string[];  
  verificationMethods?: VerificationMethod[];
```

```
budgets?: string[];
risks?: string[];

status?: EntityState;
modelType?: ModelType;
modelPhilosophy?: SpaceModelPhilosophy;

position2D?: Vector2;
position3D?: Vector3;

metadata?: Record<string, unknown>;
}
```

7. Visualização 2D

A visualização 2D é a principal prova de conceito do TG.

Deve usar React Flow com:

- ReactFlow;
- ReactFlowProvider;
- Background;
- Controls;
- MiniMap;
- fitView;
- nó customizado;
- arestas com labels;
- clique em nó;
- destaque da entidade selecionada.

Importante: importar o CSS do React Flow:

```
import "@xyflow/react/dist/style.css";
```

7.1 Níveis visuais

Organizar nós por camada vertical:

1. missão;
2. objetivos;
3. requisitos;
4. funções;
5. sistemas e segmentos;

6. subsistemas;
7. componentes;
8. interfaces;
9. budgets;
10. riscos;
11. verificações.

Se position2D existir, usar a posição do JSON. Se não existir, gerar layout automático simples.

8. Visualização 3D

A visualização 3D deve ser conceitual, não realista.

Deve usar:

- Canvas de React Three Fiber;
- OrbitControls de Drei;
- luz ambiente;
- luz direcional;
- entidades como caixas/esferas;
- interfaces como linhas;
- clique em objetos;
- destaque da seleção;
- agrupamento espacial.

Agrupamento sugerido:

- payload à frente;
- bus no centro;
- EPS em uma lateral;
- ADCS em outra lateral;
- TT&C próximo à antena;
- térmico próximo a heater/radiador;
- estação solo afastada;
- interface RF entre satélite e solo.

Se labels 3D com Text causarem erro de build, implementar sem labels ou com labels opcionais. O critério principal é o build funcionar.

9. Construção progressiva

Camadas obrigatórias:

```
[
  { id: 0, name: "Missão", types: ["mission"] },
  { id: 1, name: "Objetivos", types: ["mission", "objective"] },
  { id: 2, name: "Requisitos", types: ["mission", "objective", "requirement"] },
  { id: 3, name: "Funções", types: ["mission", "objective", "requirement", "function"] },
  { id: 4, name: "Sistema e Segmentos", types: ["mission", "objective", "requirement",
"function", "system", "segment"] },
  { id: 5, name: "Subsistemas", types: ["mission", "objective", "requirement", "function",
"system", "segment", "subsystem"] },
  { id: 6, name: "Componentes", types: ["mission", "objective", "requirement", "function",
"system", "segment", "subsystem", "component"] },
  { id: 7, name: "Interfaces", types: ["mission", "objective", "requirement", "function",
"system", "segment", "subsystem", "component", "interface"] },
  { id: 8, name: "Budgets", types: ["mission", "objective", "requirement", "function",
"system", "segment", "subsystem", "component", "interface", "budget"] },
  { id: 9, name: "Riscos", types: ["mission", "objective", "requirement", "function", "system",
"segment", "subsystem", "component", "interface", "budget", "risk"] },
  { id: 10, name: "Verificação", types: ["mission", "objective", "requirement", "function",
"system", "segment", "subsystem", "component", "interface", "budget", "risk",
"verification", "test", "operation", "model"] }
]
```

A visualização 2D e 3D deve respeitar a camada atual.

Botões:

- camada anterior;
 - próxima camada;
 - mostrar tudo;
 - reiniciar.
-

10. Painéis

10.1 Painel de detalhes

Mostrar:

- ID;
- nome;
- tipo;

- descrição;
- status;
- parentId;
- modelType;
- modelPhilosophy;
- requisitos relacionados;
- funções relacionadas;
- interfaces relacionadas;
- budgets relacionados;
- riscos relacionados;
- métodos de verificação;
- metadata formatado.

10.2 Painel de rastreabilidade

Mostrar cadeias:

Missão → Objetivo → Requisito → Função → Componente → Verificação

Gerar pelo menos três cadeias a partir do JSON demo.

10.3 Painel de budgets

Tabela:

ID | Nome | Tipo | Alocado | Estimado | Margem | Unidade | Entidade associada

Budgets mínimos:

- massa;
- potência;
- dados.

10.4 Painel de verificação

Tabela:

Requisito | Método | Item verificado | Status | Evidência

10.5 Painel de riscos

Tabela:

ID | Risco | Probabilidade | Impacto | Mitigação | Status

11. Cuidados para garantir que rode

11.1 Evitar complexidade desnecessária

Não adicionar:

- roteamento;
- backend;
- testes unitários obrigatórios;
- Tailwind;
- autenticação;
- banco;
- editor JSON completo;
- drag-and-drop avançado;
- simulação física.

11.2 Garantir build

O Claude Code deve:

1. rodar `npm run build`;
2. corrigir todos os erros de TypeScript;
3. corrigir imports quebrados;
4. corrigir problemas de tipos do Three;
5. não finalizar antes do build passar.

11.3 Padrões técnicos seguros

Usar:

- `unknown` para JSON importado antes de validar;
 - `z.infer` para derivar tipos quando conveniente;
 - `useMemo` para conversões de grafo/cena;
 - CSS puro;
 - componentes pequenos;
 - funções puras para conversão de dados;
 - fallback para posições 2D/3D ausentes;
 - fallback quando entidade referenciada não existe;
 - labels legíveis;
 - tabelas simples;
 - estado global mínimo.
-

12. Critérios de aceite

O projeto será aceite quando:

1. npm install funcionar;
 2. npm run dev abrir a aplicação;
 3. npm run build passar sem erro;
 4. a aplicação carregar o JSON demo;
 5. o status de validação aparecer;
 6. o número de entidades aparecer;
 7. o número de relações aparecer;
 8. o modo 2D mostrar grafo;
 9. o modo 3D mostrar cena;
 10. clique em entidade seleccionar;
 11. painel de detalhes atualizar;
 12. construção por camadas funcionar;
 13. rastreabilidade aparecer;
 14. budgets aparecerem;
 15. matriz de verificação aparecer;
 16. riscos aparecerem;
 17. interfaces aparecerem;
 18. o visual parecer apresentável para banca;
 19. não houver tela branca;
 20. o código estiver organizado.
-

13. Prompt final para copiar e colar no Claude Code

A próxima seção é o prompt completo. Copie do início ao fim e cole no Claude Code.

INÍCIO DO PROMPT PARA CLAUDE CODE

Você é o Claude Code atuando como engenheiro de software sênior e engenheiro de sistemas aeroespaciais.

Você está dentro da pasta *'TG-Sistema-Imersivo'*, no VS Code.

Implemente um MVP completo, funcional, robusto e apresentável para o Trabalho de Graduação em Engenharia Aeroespacial no ITA.

Título do TG

Sistema Imersivo de Modelagem e Simulação de Sistemas Espaciais em Ambiente Virtual

Objetivo geral

Criar uma aplicação web local que carregue uma arquitetura espacial descrita em JSON e permita explorá-la visualmente em 2D e 3D.

O software deve demonstrar que uma arquitetura espacial pode ser compreendida como uma rede rastreável de:

Missão → Objetivos → Requisitos → Funções → Sistema → Segmentos → Subsistemas → Componentes → Interfaces → Budgets → Riscos → Verificação → Validação

O foco é visualização sistêmica, rastreabilidade, modelagem conceitual e apoio ao entendimento de arquitetura espacial.

Não implementar simulação física real.

Fundamentação conceitual obrigatória

Considere estes princípios:

1. Engenharia de Sistemas transforma necessidades de missão em requisitos, funções, arquitetura, interfaces, budgets e verificações.
2. Requisitos devem ter rastreabilidade bidirecional com objetivos, funções, componentes e verificações.
3. A arquitetura espacial deve conter segmento espacial e segmento solo.
4. O satélite deve ser decomposto em payload e bus/service module.
5. O bus deve conter estrutura, EPS, OBC/OBDH, TT&C, ADCS, controle térmico e software embarcado.
6. Verificação confirma atendimento a requisitos especificados.
7. Validação confirma atendimento à necessidade real da missão/usuário.
8. AIV/AIT deve aparecer conceitualmente por meio de entidades de verificação, testes e evidências.
9. Product Assurance deve aparecer conceitualmente por meio de riscos, status, evidências, confiabilidade e controle de configuração.
10. O MVP deve ser uma ferramenta visual e interativa, não uma ferramenta industrial completa de MBSE.

Stack obrigatória

Use:

- Vite
- React
- TypeScript
- @xyflow/react
- three
- @react-three/fiber
- @react-three/drei
- zod
- zustand
- lucide-react
- CSS puro

Não use Tailwind, backend, banco de dados, login ou roteamento.

Se o projeto ainda não estiver inicializado, rode:

```
npm create vite@latest . -- --template react-ts
```

Depois instale:

```
npm install @xyflow/react three @react-three/fiber @react-three/drei zod zustand lucide-react
```

Garanta que funcionem:

```
npm install  
npm run dev  
npm run build
```

Não finalize antes de `npm run build` passar.

Fora do escopo

Não implemente:

- backend;
- banco de dados;
- login;
- autenticação;
- simulação orbital real;
- propagação orbital;
- dinâmica real de atitude;
- controle real de atitude;
- modelo térmico físico;

- FEA;
- CFD;
- VR completo;
- integração com hardware;
- cálculo real de link budget;
- cálculo real de power budget;
- cálculo industrial de massa.

Estrutura de arquivos obrigatória

Crie ou ajuste:

src/

App.tsx

main.tsx

index.css

data/

demoArchitecture.json

types/

architecture.ts

schemas/

architectureSchema.ts

store/

useArchitectureStore.ts

lib/

validateArchitecture.ts

progressiveBuild.ts

architectureToFlow.ts

architectureToScene.ts

traceability.ts

formatters.ts

components/

layout/

AppShell.tsx

Header.tsx

Sidebar.tsx

flow/

ArchitectureFlow.tsx
CustomNode.tsx
nodeStyles.ts

scene3d/
ArchitectureScene3D.tsx
SpaceSystemObject.tsx
InterfaceLine3D.tsx

panels/
DetailsPanel.tsx
TraceabilityPanel.tsx
BudgetPanel.tsx
VerificationPanel.tsx
RiskPanel.tsx

controls/
ViewModeToggle.tsx
BuildControls.tsx
JsonValidationStatus.tsx

Tipos TypeScript

Crie ``src/types/architecture.ts`` com:

```
``ts
export type EntityType =
  | "mission"
  | "objective"
  | "requirement"
  | "function"
  | "system"
  | "segment"
  | "subsystem"
  | "component"
  | "interface"
  | "verification"
  | "test"
  | "risk"
  | "budget"
  | "operation"
  | "model";
```

export type EntityState =

- | "draft"
- | "validated"
- | "verified"
- | "open"
- | "closed"
- | "planned"
- | "in_progress"
- | "passed"
- | "failed"
- | "waived";

export type VerificationMethod =

- | "Test"
- | "Analysis"
- | "Inspection"
- | "Review of Design"
- | "Demonstration";

export type RelationType =

- | "contains"
- | "satisfies"
- | "allocated_to"
- | "verifies"
- | "validates"
- | "depends_on"
- | "provides_power_to"
- | "sends_data_to"
- | "receives_command_from"
- | "mechanically_attached_to"
- | "thermally_coupled_to"
- | "controls"
- | "measures"
- | "actuates"
- | "communicates_with"
- | "mitigates"
- | "constrains";

export type ModelType =

- | "box"
- | "sphere"
- | "module"
- | "antenna"

```
| "solarPanel"  
| "groundStation"  
| "camera"  
| "computer"  
| "battery"  
| "sensor"  
| "actuator";
```

export type SpaceModelPhilosophy =

```
| "Engineering Model"  
| "Qualification Model"  
| "Protoflight Model"  
| "Flight Model";
```

export interface Vector2 {

```
  x: number;  
  y: number;  
}
```

export interface Vector3 {

```
  x: number;  
  y: number;  
  z: number;  
}
```

export interface ArchitectureEntity {

```
  id: string;  
  name: string;  
  type: EntityType;  
  parentId?: string;  
  description?: string;  
  
  requirements?: string[];  
  functions?: string[];  
  interfaces?: string[];  
  verificationMethods?: VerificationMethod[];  
  budgets?: string[];  
  risks?: string[];  
  
  status?: EntityState;  
  modelType?: ModelType;  
  modelPhilosophy?: SpaceModelPhilosophy;
```

```

    position2D?: Vector2;
    position3D?: Vector3;

    metadata?: Record<string, unknown>;
}

export interface ArchitectureRelation {
  id: string;
  source: string;
  target: string;
  type: RelationType;
  name?: string;
  description?: string;
  metadata?: Record<string, unknown>;
}

export interface ArchitectureModel {
  id: string;
  name: string;
  description: string;
  version: string;
  domain: "space-systems-engineering";
  entities: ArchitectureEntity[];
  relations: ArchitectureRelation[];
}

```

Schema Zod

Crie ``src/schemas/architectureSchema.ts``.

Valide:

- ArchitectureModel;
- ArchitectureEntity;
- ArchitectureRelation;
- EntityType;
- EntityStatus;
- VerificationMethod;
- RelationType;
- ModelType;
- SpaceModelPhilosophy;
- Vector2;

- Vector3;
- campos opcionais.

Crie também tipos derivados se útil.

Validação

Crie `src/lib/validateArchitecture.ts`.

A função deve retornar:

```
``ts
export interface ValidationResult {
  valid: boolean;
  entityCount: number;
  relationCount: number;
  errors: string[];
}
```

A validação deve:

1. validar o schema com Zod;
2. verificar se IDs de entidades são únicos;
3. verificar se relações apontam para entidades existentes;
4. retornar erros amigáveis;
5. nunca quebrar a aplicação.

JSON demo obrigatório

Crie `src/data/demoArchitecture.json`.

Ele deve representar uma missão de observação da Terra por satélite pequeno em LEO.

Inclua pelo menos 55 entidades e 70 relações.

A missão deve conter:

- missão;
- objetivos;
- requisitos;
- funções;
- sistema missão;
- segmento espacial;

- segmento solo;
- satélite;
- estação solo;
- payload óptico;
- bus/service module;
- estrutura;
- EPS;
- OBC/OBDH;
- TT&C;
- ADCS;
- controle térmico;
- software embarcado;
- modelos;
- componentes;
- interfaces;
- budgets;
- riscos;
- verificações;
- operações.

Use estes IDs obrigatórios:

MIS-EARTH-OBS-001
OBJ-IMG-001
OBJ-DOWNLINK-001
OBJ-SAFE-OPS-001
OBJ-TRACE-001

REQ-IMG-001
REQ-PWR-001
REQ-COMM-001
REQ-DATA-001
REQ-POINT-001
REQ-THERM-001
REQ-MASS-001
REQ-REL-001
REQ-VERIFY-001

FUN-CAPTURE-IMAGE
FUN-PROCESS-IMAGE
FUN-STORE-DATA
FUN-DOWNLINK-DATA
FUN-RECEIVE-COMMAND

FUN-GENERATE-POWER
FUN-STORE-POWER
FUN-DISTRIBUTE-POWER
FUN-DETERMINE-ATTITUDE
FUN-CONTROL-ATTITUDE
FUN-CONTROL-TEMPERATURE
FUN-EXECUTE-FSW
FUN-COMMUNICATE-GROUND

SYS-MISSION-001
SEG-SPACE-001
SEG-GROUND-001
SYS-SAT-001
SYS-GROUND-STATION-001

SUB-PAYLOAD
SUB-BUS
SUB-STRUCTURE
SUB-EPS
SUB-OBC-OBDR
SUB-TTC
SUB-ADCS
SUB-THERMAL
SUB-FSW

CMP-CAMERA
CMP-BATTERY
CMP-SOLAR-PANEL
CMP-PCDU
CMP-OBC
CMP-MASS-MEMORY
CMP-TRANSCEIVER
CMP-ANTENNA
CMP-SUN-SENSOR
CMP-MAGNETOMETER
CMP-REACTION-WHEEL
CMP-MAGNETORQUER
CMP-HEATER
CMP-RADIATOR
CMP-PRIMARY-STRUCTURE
CMP-FLIGHT-SOFTWARE
CMP-GROUND-ANTENNA
CMP-MISSION-CONTROL-SW

IF-POWER-001
IF-DATA-001
IF-CMD-001
IF-MECH-001
IF-THERM-001
IF-RF-001
IF-ADCS-DATA-001
IF-FSW-HW-001

BUD-MASS-001
BUD-POWER-001
BUD-DATA-001

RISK-POWER-001
RISK-COMM-001
RISK-THERM-001
RISK-POINT-001
RISK-DATA-001

VER-REQ-IMG-001
VER-REQ-PWR-001
VER-REQ-COMM-001
VER-REQ-POINT-001
VER-REQ-THERM-001
VER-REQ-MASS-001
VER-REQ-REL-001

TEST-FUNCTIONAL-001
TEST-TVAC-001
TEST-VIBRATION-001
TEST-RF-001
TEST-ADCS-SIM-001

OP-IMAGE-CAMPAIGN-001
OP-DOWNLINK-PASS-001
OP-SAFE-MODE-001

MOD-EM-001
MOD-QM-001
MOD-PFM-001
MOD-FM-001

Conteúdo dos requisitos

Crie requisitos com descrições conceituais:

- REQ-IMG-001: capacidade de adquirir imagens da região de interesse com resolução conceitual definida.
- REQ-PWR-001: geração e armazenamento de potência com margem.
- REQ-COMM-001: downlink de dados e recepção de telecomandos.
- REQ-DATA-001: armazenamento de dados de payload.
- REQ-POINT-001: apontamento adequado para aquisição de imagem.
- REQ-THERM-001: manter componentes dentro da faixa térmica operacional.
- REQ-MASS-001: manter massa total dentro de limite conceitual.
- REQ-REL-001: manter confiabilidade conceitual para a missão.
- REQ-VERIFY-001: requisitos críticos devem possuir método de verificação definido.

Use campo ``metadata`` para guardar parâmetros conceituais.

Conteúdo de budgets

Entidades do tipo budget devem ter metadata:

```
{
  "budgetType": "mass | power | data",
  "allocated": número,
  "estimated": número,
  "margin": número,
  "unit": "kg | W | MB | Mbps",
  "associatedEntity": "ID da entidade"
}
```

Conteúdo de riscos

Entidades do tipo risk devem ter metadata:

```
{
  "probability": "low | medium | high",
  "impact": "low | medium | high",
  "mitigation": "texto",
  "owner": "subsystem ou função",
  "residualRisk": "low | medium | high"
}
```

Conteúdo de verificações

Entidades do tipo verification devem ter metadata:

```
{
  "requirementId": "REQ-...",
  "verifiedItemId": "CMP-... ou SUB-...",
  "method": "Test | Analysis | Inspection | Review of Design | Demonstration",
  "status": "planned | in_progress | passed | failed | waived",
  "evidence": "descrição curta da evidência esperada",
  "verificationLevel": "component | subsystem | system"
}
```

Relações obrigatórias

Inclua relações que representem:

- missão contém objetivos;
- objetivos satisfazem missão;
- requisitos satisfazem objetivos;
- funções são alocadas a requisitos;
- componentes são alocados a funções;
- subsistemas contêm componentes;
- satélite contém subsistemas;
- segmento espacial contém satélite;
- segmento solo contém estação solo;
- interfaces conectam componentes/subsistemas;
- budgets restringem subsistemas;
- riscos afetam componentes/subsistemas;
- verificações verificam requisitos;
- testes implementam verificações;
- operações validam objetivos.

Crie relações suficientes para que a visualização não fique vazia nem pobre.

Construção progressiva

Crie ``src/lib/progressiveBuild.ts``.

Camadas:

```
``ts
export const BUILD_LAYERS = [
  { id: 0, name: "Missão", types: ["mission"] },
  { id: 1, name: "Objetivos", types: ["mission", "objective"] },
```

```

    { id: 2, name: "Requisitos", types: ["mission", "objective", "requirement"] },
    { id: 3, name: "Funções", types: ["mission", "objective", "requirement", "function"] },
    { id: 4, name: "Sistema e Segmentos", types: ["mission", "objective", "requirement",
"function", "system", "segment"] },
    { id: 5, name: "Subsistemas", types: ["mission", "objective", "requirement", "function",
"system", "segment", "subsystem"] },
    { id: 6, name: "Componentes", types: ["mission", "objective", "requirement", "function",
"system", "segment", "subsystem", "component"] },
    { id: 7, name: "Interfaces", types: ["mission", "objective", "requirement", "function",
"system", "segment", "subsystem", "component", "interface"] },
    { id: 8, name: "Budgets", types: ["mission", "objective", "requirement", "function",
"system", "segment", "subsystem", "component", "interface", "budget"] },
    { id: 9, name: "Riscos", types: ["mission", "objective", "requirement", "function", "system",
"segment", "subsystem", "component", "interface", "budget", "risk"] },
    { id: 10, name: "Verificação e Operações", types: ["mission", "objective", "requirement",
"function", "system", "segment", "subsystem", "component", "interface", "budget", "risk",
"verification", "test", "operation", "model"] }
] as const;
`

```

Crie funções:

- getVisibleTypesForLayer
- filterEntitiesByLayer
- filterRelationsByVisibleEntities
- getLayerName
- getMaxLayer

Store Zustand

Crie `src/store/useArchitectureStore.ts`.

Estado:

```

``ts
architecture: ArchitectureModel;
selectedEntityId: string | null;
viewMode: "2d" | "3d";
currentLayer: number;
validationResult: ValidationResult;
`

```

Actions:

```

``ts
selectEntity(id: string | null): void;
setViewMode(mode: "2d" | "3d"): void;
toggleViewMode(): void;
nextLayer(): void;
previousLayer(): void;
showAllLayers(): void;
resetLayers(): void;
setArchitecture(model: ArchitectureModel): void;
``

```

Seletores/funções úteis:

- getSelectedEntity;
- getVisibleEntities;
- getVisibleRelations.

Conversão para React Flow

Crie ``src/lib/architectureToFlow.ts``.

Função:

```

``ts
architectureToFlow(
  entities: ArchitectureEntity[],
  relations: ArchitectureRelation[],
  selectedEntityId: string | null
): { nodes: Node[]; edges: Edge[] }
``

```

Regras:

- usar position2D quando existir;
- gerar posição automática quando não existir;
- ordenar por tipo;
- labels nas arestas;
- tipo da aresta pode ser smoothstep;
- nodes devem usar custom node;
- passar dados do nó: id, name, type, description, status, selected.

Visualização 2D

Crie:

- ``src/components/flow/ArchitectureFlow.tsx``
- ``src/components/flow/CustomNode.tsx``
- ``src/components/flow/nodeStyles.ts``

Obrigatório:

```
``ts
import "@xyflow/react/dist/style.css";
``
```

Usar:

- ReactFlow;
- ReactFlowProvider;
- Background;
- Controls;
- MiniMap.

Funcionalidades:

- clicar em nó seleciona entidade;
- clicar no fundo deseleciona;
- nó selecionado fica destacado;
- cores por tipo;
- labels nas arestas;
- visual legível;
- não quebrar se grafo estiver vazio.

Conversão para cena 3D

Crie ``src/lib/architectureToScene.ts``.

Gerar:

```
``ts
SceneEntity {
  id;
  name;
  type;
  modelType;
  position;
  color;
```

```

    scale;
}

SceneRelation {
  id;
  source;
  target;
  type;
  sourcePosition;
  targetPosition;
}
...

```

Usar posição3D do JSON ou fallback automático.

Visualização 3D

Crie:

```

- `src/components/scene3d/ArchitectureScene3D.tsx`
- `src/components/scene3d/SpaceSystemObject.tsx`
- `src/components/scene3d/InterfaceLine3D.tsx`

```

Usar:

- Canvas;
- OrbitControls;
- ambientLight;
- directionalLight;
- gridHelper ou equivalente se não causar erro;
- meshes simples;
- linhas para relações/interfaces;
- clique em objeto seleciona entidade;
- objeto selecionado fica com emissive ou escala maior.

Formas:

- mission: sphere;
- objective: sphere;
- requirement: box;
- function: box;
- system: large box;
- segment: box;

- subsystem: box;
- component: small box;
- interface: line;
- budget: box;
- risk: sphere;
- verification: sphere;
- test: sphere;
- operation: box;
- model: box.

Se labels 3D causarem erro de build, remova labels 3D. O build funcionando é prioridade.

Cores por tipo

Crie mapa de cores:

mission: #38bdf8
objective: #22d3ee
requirement: #facc15
function: #c084fc
system: #4ade80
segment: #86efac
subsystem: #60a5fa
component: #e5e7eb
interface: #fb923c
budget: #a3e635
verification: #f472b6
test: #f9a8d4
risk: #f87171
operation: #ffffff
model: #818cf8

Painel de detalhes

Crie ``src/components/panels/DetailsPanel.tsx``.

Se não houver entidade selecionada, mostrar:

“Selecione uma entidade na visualização 2D ou 3D para inspecionar seus detalhes.”

Também mostrar a cadeia do TG:

Missão → Objetivos → Requisitos → Funções → Componentes → Verificação

Se houver entidade selecionada, mostrar:

- ID;
- nome;
- tipo;
- descrição;
- status;
- parentId;
- modelType;
- modelPhilosophy;
- requirements;
- functions;
- interfaces;
- budgets;
- risks;
- verificationMethods;
- metadata formatado com `JSON.stringify(metadata, null, 2)`.

Rastreabilidade

Crie ``src/lib/traceability.ts``.

Gerar cadeias com base em relações e metadata.

Interface sugerida:

```
``ts
export interface TraceabilityChain {
  mission?: ArchitectureEntity;
  objective?: ArchitectureEntity;
  requirement?: ArchitectureEntity;
  function?: ArchitectureEntity;
  component?: ArchitectureEntity;
  verification?: ArchitectureEntity;
}
```

Crie função:

```
``ts
export function buildTraceabilityChains(model: ArchitectureModel):
TraceabilityChain[]
```

...

A função deve tentar montar cadeias:

mission → objective → requirement → function → component → verification

Usar relações:

- satisfies;
- allocated_to;
- verifies;
- contains;
- depends_on.

Se faltar alguma relação, usar metadata ``requirementId`` e ``verifiedItemId``.

Mostrar pelo menos três cadeias no painel.

Crie ``src/components/panels/TraceabilityPanel.tsx``.

BudgetPanel

Crie ``src/components/panels/BudgetPanel.tsx``.

Tabela:

ID | Nome | Tipo | Alocado | Estimado | Margem | Unidade | Entidade associada

Extrair de entidades ``type === "budget"`` e metadata.

VerificationPanel

Crie ``src/components/panels/VerificationPanel.tsx``.

Tabela:

Requisito | Método | Item verificado | Status | Evidência

Extrair de entidades ``type === "verification"`` e metadata.

Status visual:

- planned: cinza/azul;
- in_progress: amarelo;

- passed: verde;
- failed: vermelho;
- waived: roxo.

RiskPanel

Crie ``src/components/panels/RiskPanel.tsx``.

Tabela:

ID | Risco | Probabilidade | Impacto | Mitigação | Status

Extrair de entidades ``type === "risk"`` e metadata.

Controles

Crie:

- ViewModeToggle.tsx
- BuildControls.tsx
- JsonValidationStatus.tsx

ViewModeToggle:

- botão 2D;
- botão 3D;
- estado ativo.

BuildControls:

- mostrar camada atual;
- camada anterior;
- próxima camada;
- mostrar tudo;
- reiniciar.

JsonValidationStatus:

- válido/inválido;
- contagem de entidades;
- contagem de relações;
- lista curta de erros.

Layout

Crie:

- AppShell.tsx
- Header.tsx
- Sidebar.tsx

Layout:

- Header no topo;
- área principal em grid;
- visualização central;
- painel lateral direito;
- painel inferior com rastreabilidade, budgets, verificação, riscos.

Pode usar seções empilhadas no painel inferior. Não precisa implementar abas sofisticadas.

CSS

Crie `src/index.css`.

Tema:

- fundo escuro;
- painéis cinza escuro;
- bordas discretas;
- texto claro;
- ciano/azul como destaque;
- tabelas legíveis;
- botões modernos;
- layout responsivo para notebook;
- altura total da viewport.

Não deixar tela branca.

README

Crie `README.md` com:

- título do TG;
- descrição do MVP;
- stack;
- como rodar;

- o que está implementado;
- fora do escopo;
- próximos passos.

Integração no App

``src/App.tsx`` deve apenas renderizar ``AppShell``.

``src/main.tsx`` deve importar ``index.css`` e renderizar App.

Regras de robustez

1. Não use ``any`` se puder usar ``unknown`` ou tipos específicos.
2. Não quebre se houver entidade referenciada ausente.
3. Não quebre se metadata estiver vazio.
4. Não quebre se posição 2D/3D estiver ausente.
5. Não finalize com erro de TypeScript.
6. Não finalize com erro de build.
7. Não adicione dependências desnecessárias.
8. Não implemente funcionalidades fora do escopo.
9. Não use Tailwind.
10. Não implemente backend.

Ordem de execução

Siga exatamente:

1. Inspecione a pasta atual.
2. Se não houver projeto Vite, inicialize Vite React TypeScript.
3. Instale dependências.
4. Crie tipos.
5. Crie schema.
6. Crie JSON demo.
7. Crie validação.
8. Crie progressiveBuild.
9. Crie store Zustand.
10. Crie formatters.
11. Crie conversão para React Flow.
12. Crie visualização 2D.
13. Crie conversão para cena 3D.
14. Crie visualização 3D.
15. Crie traceability.
16. Crie painéis.

17. Crie controles.
18. Crie layout.
19. Ajuste CSS.
20. Crie README.
21. Rode ``npm run build``.
22. Corrija todos os erros.
23. Ao final, liste arquivos criados/alterados e explique como rodar.

Critérios de aceite

O projeto só estará pronto se:

- npm install funciona;
- npm run dev funciona;
- npm run build passa;
- a arquitetura demo aparece;
- JSON válido aparece;
- contagem de entidades aparece;
- contagem de relações aparece;
- modo 2D funciona;
- modo 3D funciona;
- clique em nó 2D seleciona entidade;
- clique em objeto 3D seleciona entidade;
- painel de detalhes mostra dados;
- construção progressiva funciona;
- rastreabilidade mostra pelo menos três cadeias;
- budgets aparecem;
- verificações aparecem;
- riscos aparecem;
- interfaces aparecem;
- visual é apresentável para banca;
- não há tela branca;
- não há erros no console relacionados a imports quebrados.

Implemente agora.

FIM DO PROMPT PARA CLAUDE CODE

14. Como usar no VS Code

1. Abra o VS Code na pasta TG-Sistema-Imersivo.

2. Abra o Claude Code.
3. Cole o prompt da seção anterior.
4. Deixe o Claude executar em etapas.
5. Aprove apenas comandos coerentes:
 - npm create vite@latest . --template react-ts;
 - npm install ...;
 - criação/alteração de arquivos;
 - npm run build.
6. Se o Claude parar antes de rodar build, peça:
“Rode npm run build e corrija todos os erros antes de finalizar.”
7. Quando terminar, rode:

npm install

npm run dev

8. Abra o endereço local indicado, normalmente:
http://localhost:5173
-

15. Próximos passos após o MVP

Depois que o MVP rodar, os próximos incrementos ideais são:

1. editor JSON no navegador;
 2. upload de arquivo JSON;
 3. exportação de imagem do grafo;
 4. filtros por tipo de entidade;
 5. busca por ID/nome;
 6. destaque de caminho de rastreabilidade ao selecionar requisito;
 7. layout automático melhorado;
 8. geração de matriz de verificação em CSV;
 9. suporte a múltiplas arquiteturas;
 10. README acadêmico com metodologia e resultados esperados;
 11. capítulo de implementação no TG;
 12. trabalhos futuros: VR, orbital, térmico, FEA, banco de dados, backend, integração MBSE.
-

16. Referências técnicas usadas como base conceitual

Este prompt foi estruturado a partir de conceitos consolidados de Engenharia de Sistemas e sistemas espaciais, especialmente:

- NASA Systems Engineering Handbook, NASA/SP-2007-6105 Rev1.

- NASA Systems Engineering Handbook, SP-6105, 1995.
- Spacecraft Systems Engineering, Fourth Edition, Fortescue, Swinerd & Stark, 2011.

Os pontos incorporados incluem:

- decomposição de missão, requisitos, arquitetura e produto;
- importância de rastreabilidade;
- decomposição lógica e funcional;
- arquitetura como relações entre elementos;
- payload e bus/service module;
- subsistemas típicos de spacecraft;
- budgets de massa, potência e dados;
- interface management;
- verificação e validação;
- AIV/AIT;
- product assurance e risco;
- necessidade de visualização clara para explorar sistemas complexos.